SQL JOINS

The Complete Interview Guide

1. What is a JOIN?

A JOIN is used to combine rows from two or more tables based on a related column between them.

2. Why do we use JOINs?

- ✅ To retrieve data from multiple tables in a single query.
- ✅ To avoid redundant data.
- ✅ To get meaningful insights by combining related information.
- ✅ To solve real-world business problems efficiently.

Why JOINs are Important?

- Most real-world data is distributed across multiple tables.
- JOINs help us connect the dots between different datasets.
- More than 70% of SQL interview questions involve JOINs.

3. Types of JOINs

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- SELF JOIN

4. Real-World Example

Let's understand JOINs using two simple tables.

Table: Customers

CustomerID	CustomerName	City
1	Rohan	Delhi
2	Priya	Mumbai
3	Amit	Bangalore
4	Neha	Hyderabad

Link: Customers.CustomerID = Orders.CustomerID

Table: Orders

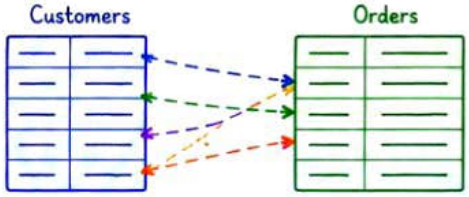
OrderID	CustomerID	OrderDate	Amount
101	1	2024-01-10	1500
102	2	2024-01-12	2500
103	1	2024-01-15	3000
104	3	2024-01-18	1200

How JOINs Work?

JOINs match rows from both tables based on a related column.
Example: Match CustomerID from Customers table with CustomerID from Orders table.

JOIN returns matching rows from both tables.

Visual Understanding



JOIN combines matching rows to produce meaningful results.

Interview Tip

Always check the relationship between tables before applying any JOIN.
Understand the data first, then write the query!

SQL JOINS

INNER JOIN

INNER JOIN returns only the matching rows from both tables based on the related column.

1. Example Tables

Customers

CustomerID	CustomerName	City
1	Rohan	Delhi
2	Priya	Mumbai
3	Amit	Bangalore
4	Neha	Hyderabad

Orders

OrderID	CustomerID	OrderDate	Amount
101	1	2024-01-10	1500
102	2	2024-01-12	2500
103	1	2024-01-15	3000
104	3	2024-01-18	1200
105	5	2024-01-20	2000

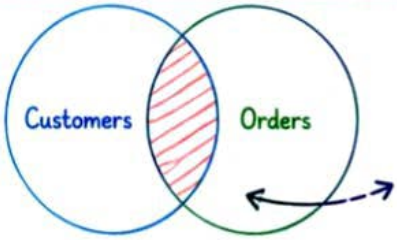
Key Point

Only those customers who have placed orders will appear in the result.

2. INNER JOIN Syntax

```
SELECT  c.CustomerID, c.CustomerName, c.City,
        o.OrderID, o.OrderDate, o.Amount
FROM    Customers c
INNER JOIN Orders o
ON      c.CustomerID = o.CustomerID;
```

3. Visual Representation



INNER JOIN returns only the matching records (intersection)

4. INNER JOIN Output

CustomerID	CustomerName	City	OrderID	OrderDate	Amount
1	Rohan	Delhi	101	2024-01-10	1500
1	Rohan	Delhi	103	2024-01-15	3000
2	Priya	Mumbai	102	2024-01-12	2500
3	Amit	Bangalore	104	2024-01-18	1200

What Happened?

- ✅ CustomerID = 1 → 2 orders
- ✅ CustomerID = 2 → 1 order
- ✅ CustomerID = 3 → 1 order
- ✅ CustomerID = 4 → No order
  → So, not shown in result.

5. When to Use INNER JOIN?

- When you only need matching data from both tables.
- When unmatched rows are not required.
- When building reports with only relevant relationships.

6. Example Interview Question

Q. Write a query to find all customers who have placed at least one order.

A. Use INNER JOIN between Customers and Orders on CustomerID.

SQL JOINS

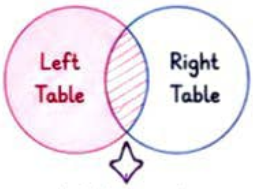
LEFT JOIN vs RIGHT JOIN

LEFT JOIN returns all rows from the LEFT table and matching rows from the RIGHT table. RIGHT JOIN returns all rows from the RIGHT table and matching rows from the LEFT table.

LEFT JOIN (Keep All Left Records)

1. Syntax

```
SELECT columns
FROM left_table L
LEFT JOIN right_table R
ON L.key = R.key;
```



All from Left
+ Matching from Right

2. Example

Customers (C)

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit
4	Neha

Orders (O)

OrderID	CustomerID
101	1
102	2
103	1
104	5

CustomerID 5 does not exist in Customers

3. LEFT JOIN Output (C LEFT JOIN O)

CustomerID	CustomerName	OrderID
1	Rohan	101
1	Rohan	103
2	Priya	102
3	Amit	NULL
4	Neha	NULL

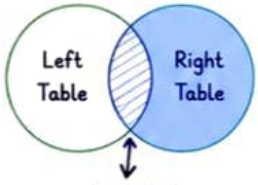
All customers are shown. Orders are shown if available, otherwise NULL.

VS

RIGHT JOIN (Keep All Right Records)

1. Syntax

```
SELECT columns
FROM left_table L
RIGHT JOIN right_table R
ON L.key = R.key;
```



All from Right
+ Matching from Left

2. Example

Customers (C)

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit
4	Neha

Orders (O)

OrderID	CustomerID
101	1
102	2
103	1
104	5

CustomerID 5 does not exist in Customers

3. RIGHT JOIN Output (C RIGHT JOIN O)

CustomerID	CustomerName	OrderID
1	Rohan	101
1	Rohan	103
2	Priya	102
5	NULL	104

All orders are shown. Customers are shown if available, otherwise NULL.

4. When to Use?

Use LEFT JOIN when:

- You want all records from the left table.
- You want to find non-matching records in the right table.

Use RIGHT JOIN when:

- You want all records from the right table.
- You want to find non-matching records in the left table.

5. Common Mistakes

- Confusing LEFT JOIN and RIGHT JOIN.
- Applying filters on the right table in WHERE clause (turns LEFT JOIN into INNER JOIN).
- Not handling NULL values in results.
- Not understanding which table is preserved in the result

6. Interview Question

Q. Find all customers and their orders. Also include customers who have not placed any order.

A.

```
SELECT C.CustomerID, C.CustomerName, O.OrderID
FROM Customers C
LEFT JOIN Orders O
ON C.CustomerID = O.CustomerID;
```

Pro Tip

LEFT JOIN keeps all left rows, RIGHT JOIN keeps all right rows. Remember: L stays, R may go NULL!

Key Takeaway

LEFT JOIN and RIGHT JOIN are opposite of each other. Understand which table you want to "preserve" in the result.

Practice more questions to master JOINs!

SQL JOINS

FULL OUTER JOIN & CROSS JOIN

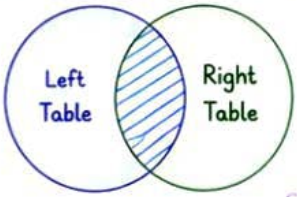
These joins help us return all records from both tables (FULL OUTER JOIN) or create combinations of rows (CROSS JOIN).

1. FULL OUTER JOIN

FULL OUTER JOIN returns all rows when there is a match in either left or right table. If there is no match, NULL is returned for the missing side.

Syntax

```
SELECT columns
FROM left_table L
FULL OUTER JOIN right_table R
ON L.key = R.key;
```



All from Left
+ All from Right

2. Example

Customers (C)

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit
4	Neha

Orders (O)

OrderID	CustomerID
101	1
102	2
103	1
104	5

3. FULL OUTER JOIN Output (C FULL OUTER JOIN O)

CustomerID	CustomerName	OrderID	CustomerID
1	Rohan	101	1
1	Rohan	103	1
2	Priya	102	2
3	Amit	NULL	NULL
4	Neha	NULL	NULL
NULL	NULL	104	5

All customers are shown.
All orders are shown.
Non-matching rows have NULL.

When to Use?

- When you want all data from both tables.
- When analyzing data completeness.
- Useful in reconciliation & comparison reports.

4. CROSS JOIN

CROSS JOIN returns the Cartesian Product of both tables. Every row from the left table is combined with every row from the right table.

Syntax

```
SELECT columns
FROM left_table
CROSS JOIN right_table;
```

5. Example

Customers (C)

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit

Orders (O)

OrderID	CustomerID
101	1
102	2

6. CROSS JOIN Output (C CROSS JOIN O)

CustomerID	CustomerName	OrderID	CustomerID
1	Rohan	101	1
1	Rohan	102	2
2	Priya	101	1
2	Priya	102	2
3	Amit	101	1
3	Amit	102	2

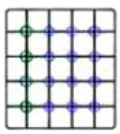
3 rows from Customers × 2 rows from Orders = 6 rows in result.

Key Difference

FULL OUTER JOIN

Returns all rows from both tables.

CROSS JOIN



Returns all possible combinations of rows.

Common Mistake

- Using CROSS JOIN by mistake (it can create huge result sets).
- Not adding proper conditions with FULL OUTER JOIN.

Pro Tip

Always apply filters and proper conditions when using FULL OUTER JOIN.
Be careful with CROSS JOIN on large tables!

Interview Tip

Understand the business requirement carefully before choosing the type of JOIN.

SQL JOINS

SELF JOIN

A SELF JOIN is a regular JOIN, but the table is joined with itself. It is useful for comparing rows within the same table.

1. When to Use SELF JOIN?

- Hierarchical data (Employee-Manager, Category-Subcategory)
- Finding relationships within the same table
- Comparing rows of the same entity

2. Example Scenario

We have an Employees table.
We want to display each employee along with their Manager Name.

Employees Table

EmpID	EmpName	ManagerID
1	Ratan	NULL
2	Neha	1
3	Amit	1
4	Priya	2
5	Kunal	2
6	Sneha	3

3. SELF JOIN Syntax

```
SELECT  e.EmpID, e.EmpName,
        m.EmpName AS ManagerName
FROM    Employees e
LEFT JOIN Employees m
ON      e.ManagerID = m.EmpID;
```

Key Point

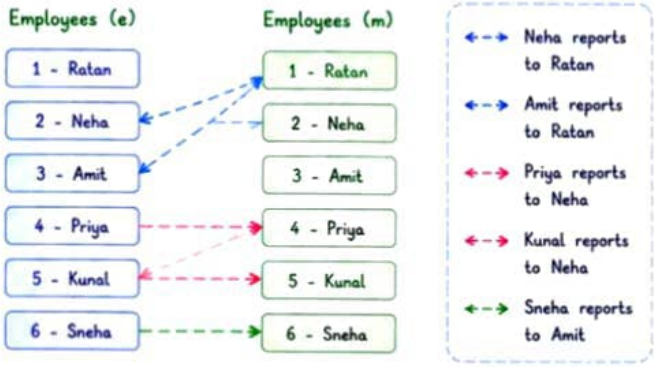
We use different aliases (e, m) for the same table.

4. SELF JOIN Output

EmpID	EmpName	ManagerID	ManagerName
1	Ratan	NULL	NULL
2	Neha	1	Ratan
3	Amit	1	Ratan
4	Priya	2	Neha
5	Kunal	2	Neha
6	Sneha	3	Amit

Ratan has no manager, so ManagerName is NULL.

5. Visual Representation



6. Interview Questions

- Q1. Find all employees and their managers.
- Q2. Find employees who don't have a manager.
- Q3. Find the number of employees managed by each manager.
- Q4. Find managers who have more than 2 direct reports.

7. Common Mistakes

- Not using aliases for the same table.
- Using INNER JOIN - it will remove employees without managers.
- Confusing EmpID and ManagerID.
- Not handling NULL values.

8. Pro Tip

- Use LEFT JOIN to include employees who don't have managers.
- Always test your query with a small dataset.
- SELF JOIN is widely used in hierarchies, reporting & organizational data.

Key Takeaway

SELF JOIN helps us relate rows within the same table.
It is essential for hierarchical data and is frequently asked in SQL interviews.

Practice more to master SELF JOIN!

SQL JOINS

INTERVIEW QUESTIONS

Practice these popular SQL interview questions to strengthen your JOIN skills.

1. Customers who never placed an order

Write a query to find customers who have not placed any order.

Customers

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit
4	Neha

Orders

OrderID	CustomerID
101	1
102	2
103	1

Hint:
Use LEFT JOIN and filter where OrderID is NULL.

2. Employees with their Managers

Write a query to display each employee along with their manager's name.

Employees

EmpID	EmpName	ManagerID
1	Ratan	NULL
2	Neha	1
3	Amit	1
4	Priya	2
5	Kunal	2

Tip

This is a SELF JOIN example using the same table.

3. Products that were never sold

Write a query to find products that do not appear in any order.

Products

ProductID	ProductName
1	Laptop
2	Mobile
3	Tablet
4	Headphones

OrderDetails

OrderID	ProductID
101	1
102	2
103	1

Hint: Use LEFT JOIN from Products to OrderDetails.

4. Highest selling product in each category

Write a query to find the highest selling product within each category.

Products

ProductID	ProductName	CategoryID
1	Laptop	1
2	Mobile	1
3	Tablet	2
4	TV	2

Sales

ProductID	TotalSales
1	50000
2	80000
3	30000
4	70000

Hint: Use JOIN, GROUP BY and aggregate functions.

5. Total sales by each customer

Write a query to find total sales amount for each customer.

Customers

CustomerID	CustomerName
1	Rohan
2	Priya
3	Amit

Orders

OrderID	CustomerID	Amount
101	1	1500
102	1	2500
103	2	3000
104	3	1200

Hint: Use JOIN and GROUP BY on CustomerID.

Key Takeaways

- JOINs combine rows from two or more tables.
- Different JOINs are used based on the requirement.
- Always check which table should be preserved.
- Understand the data and then write the query.

Common Mistakes to Avoid

- Using INNER JOIN when LEFT JOIN is required.
- Forgetting ON condition.
- Using WHERE clause on right table in LEFT JOIN.
- Not considering NULL values in results.

Quick Recap (JOIN Types)

- INNER JOIN – Matching rows only
- LEFT JOIN – All left + matching right
- RIGHT JOIN – All right + matching left
- FULL OUTER JOIN – All from both tables
- CROSS JOIN – Cartesian product
- SELF JOIN – Join a table with itself

Remember!

Good SQL is not about writing complex queries, it's about writing the right query in the right way.